

CyberSuite

CyberSuite is a port scanner platform with multiple services running like port scanner, subdomain finder, password checker etc, The cybersuite is basically targetting to make things easier for everybody to check for ip address there passwords and everything this website does

CyberSuite can be used by from any Professional or the person from the Cyber security field to test and run the website for the simulatory work

CyberSuite is made up of:

1. The Backend (API & Core Services) The backend is written entirely in **Python (3.11 / 3.12)** and built around an asynchronous, high-performance architecture.

- **Web Framework: FastAPI** — Used for building the core REST API because of its speed, asynchronous capabilities, and automatic documentation (Swagger UI).
- **Application Server: Uvicorn** — A lightning-fast ASGI web server running the FastAPI app.
- **Data Validation: Pydantic & pydantic-settings** — Used strictly for typing, validating incoming API requests, and managing environment variables securely.
- **Rate Limiting / Security: Slowapi** — Protects the API endpoints from spam and brute-force requests by rate-limiting IPs.
- **Core Security Services & Modules: Networking / Port Scanning:** Custom Python socket programming (socket, ssl) and concurrent.futures for asynchronous, multi-threaded port and network scanning.

- **Subdomain / DNS Lookups:** Powered by **dnspython** for querying various DNS records (A, MX, TXT) and brute-forcing subdomains.
- **WHOIS Lookups:** Built using the **python-whois** library to query domain registration databases.
- **Web Scanners & Exploits:** Utilizing **HTTPX** (an asynchronous HTTP client) to perform web vulnerability assessments, SQLi / XSS simulated tests, and WordPress enumeration without blocking the server.
- **AI Integration Service:** Powered by the **OpenAI Python SDK** (openai) to drive the internal "AI Assistant" for interpreting logs or providing security recommendations.
- **Reporting Engine:** Built with **ReportLab** (reportlab) to dynamically generate downloadable PDF security reports from scan results.
- **AI using Grok:** Built the ai assistant using the grok api of llama-3.3-70b-versatile, llama-3.1-8b-instant, mixtral-8x7b-32768 which helps the person to fine answers.
- **Email Headers:** Integrated the Email headers for getting the full information about the email that has been sent to you.
-

2. The Frontend (User Interface) The frontend is a modern Single Page Application (SPA) written in **JavaScript / JSX**.

- **Core Library: React 18** — The foundation of the user interface.
- **Build Tool: Vite** — Replaces older tools like Webpack or Create React App to provide ultra-fast Hot Module Replacement (HMR) and optimized production builds.
- **Routing: React Router** (react-router-dom) — Manages the multi-

page navigation (Dashboard, Recon Tools, AI Assistant, Email Analyzer, etc.) seamlessly without reloading the browser.

- **State Management: Zustand** — A lightweight, fast state manager used (e.g., in authStore.js) for keeping track of user authentication and global application states.
- **HTTP Requests: Axios** — Used to securely communicate with the FastAPI backend.
- **UI / Styling & Design: Tailwind CSS & PostCSS** — The core utility framework used to build the responsive layout.
- **Custom CSS (globals.css):** Features highly customized, hand-written design tokens to achieve the "Cinematic Cyber UI", including dark glassmorphism effects (.glass), neon text glows, fluid responsive typography, animated backgrounds, and custom scrollbars.
- **Animations & Visuals: Framer Motion** — Used heavily for smooth page transitions, sliding mobile menus, and interactive component animations.
- **Three.js** (three) — WebGL 3D rendering library, likely used for complex background animations or rendering the 3D Attack Map globe.
- **Chart.js & react-chartjs-2** — Used for rendering beautiful data visualizations and analytics graphs on the Dashboard.
- **Icons & Feedback: React Icons** (react-icons/fi - Feather Icons) — Provides the sleek, minimalist iconography throughout the app.
- **React Hot Toast** — Handles the sleek popup notifications (success/error alerts) that appear in the bottom-right corner.

These are things that have been used for making the Cybersuite more

perfect and the next level

Problems I faced

- 1) *Backend kept crashing during port scans* — learned about socket timeouts, ThreadPoolExecutor limits, and asyncio concurrency management
- 2) *Python version chaos* — had 3 different Python versions (3.11, 3.12, 3.14) fighting each other on Windows. Learned the hard way why virtual environments exist
- 3) *bcrypt version conflict with passlib* — newer bcrypt removed `__about__` attribute that passlib depends on. Fixed by pinning `bcrypt==4.0.1`
- 4) *Groq model decommissioned mid-build* — llama3-70b-8192 was retired while I was building. Had to migrate to llama-3.3-70b-versatile
- 5) *API routes returning 404* — frontend was calling `/api/recon/port` but backend registered it as `/api/recon/port-scan`. Learned to always verify endpoint names in both places
- 6) *AI giving pre-saved answers* — replaced hardcoded responses with real Groq API calls, added conversation history, model selector, and live token counter
- 7) *SSL checker showing Unknown for expired certs* — learned that Python's ssl module skips cert parsing on verification failure, fixed using `CERT_NONE` mode for data extraction

What Makes This Different From a Typical Student Project:

- All tools work on real websites — not just localhost demos
- Legal disclaimer system with per-tool capability labels
- Rate limiting on every endpoint to prevent abuse
- JWT authentication with proper bcrypt hashing
- Real AI that understands cybersecurity context

→ PDF reports you can actually download and share

Still Improving:

→ Deploying on Railway (backend) + Vercel (frontend)

→ Adding CVE lookup with NVD API integration

→ Improving SQLi/XSS payload coverage

→ AI scan result analysis — feed scan output directly to Groq for recommendations

This project taught me more about real-world development than any tutorial ever could. Every bug was a lesson.